API Documentation

MediCare.API — Proposed Backend REST Contract

API-FIRST ARCHITECTURE | ASP.NET CORE | MICROSOFT AZURE

Document Control

Project	MediCare — Clinic Management & Appointment System
Phase	Phase 5 — Deployment & Integration
Architecture	API-First N-Tier Web Application
Version	1.0 — Proposed baseline

How to Read This Document

Items marked CONFIRMED are explicit project decisions or requirements. Items marked PROPOSED are recommended technical choices that require approval or implementation. Items marked OPTIONAL depend on scope and schema approval. Items marked OPEN require a team decision before the related implementation is treated as final.

1. Purpose and Status

This document defines the proposed REST API contract for MediCare.API. It is derived from the supplied functional requirements, business rules and logical database schema. No source-code repository, Controllers, DTOs, .csproj files or live OpenAPI document were supplied for this task; therefore every route below is a proposed contract and must be reconciled with the implementation before it is called an implemented endpoint.

Architecture rule — MediCare.Web is an ASP.NET Core MVC presentation client. It calls MediCare.API through a server-side Typed HttpClient. The MVC application never connects directly to SQL Server and never owns authoritative business rules.

2. API Conventions

Item	Convention
Base path	/api/v1
Transport	HTTPS only; JSON request/response payloads
Naming	Plural resource names; lowercase kebab-free resource paths; action subroutes only for domain commands such as /confirm and /cancel
Authentication	Proposed JWT Bearer for MVC-to-API requests; confirm token lifecycle and Identity configuration before implementation
Authorization	Roles Patient, Doctor, Admin plus resource ownership and doctor-patient relationship checks
Pagination	page (1-based) and pageSize; enforce a server-side maximum such as 100
Time handling	Accept ISO 8601 timestamps with offset; normalize/persist UTC; render Cairo local time in UI
Concurrency	Return 409 Conflict for slot overlap, stale state, duplicate medical record, or prescription version conflict
Errors	Use a consistent Problem Details-based error envelope; never return stack traces or sensitive medical data
Versioning	URL versioning /api/v1 is proposed; maintain backwards compatibility within v1

3. Authentication and MVC Session Flow

- Patient and Doctor can submit public registration requests. Doctor accounts remain Pending until Admin approval. Admin registration is never public; seed/provision the initial Admin securely.
- MediCare.API validates credentials using ASP.NET Core Identity. JWT Bearer is the proposed API scheme; this is not confirmed as implemented because source code was not supplied.
- MediCare.Web creates a secure HttpOnly, Secure, appropriately SameSite browser session. API credentials are kept server-side in protected storage and attached by Typed HttpClient. Do not store long-lived API tokens in localStorage or URL parameters.
- The API must re-check the role, doctor approval status, resource owner and business state on every sensitive operation. UI checks are convenience only.
- For direct browser SignalR connections, use the trusted API negotiation endpoint and a short-lived hub-scoped access token. Keep it in memory only; never send medical details in event payloads.

4. Roles and Authorization Matrix

Module / Operation	Anonymous	Patient	Doctor	Admin
Patient / Doctor registration	Allowed	—	—	Admin may manage
Doctor search / public profile	Allowed (approved/active only)	Allowed	Allowed	Allowed
Book appointment	No	Own booking only	No patient booking unless explicit business rule	Admin-managed action only
Cancel / reschedule	No	Own appointment + >24h	No	Authorized administrative action

Module / Operation	Anonymous	Patient	Doctor	Admin
Confirm / reject	No	No	Assigned doctor + Pending only	Administrative override only if approved
Medical record create	No	No	Assigned doctor + eligible visit	No routine clinical authoring
Medical record read	No	Own only	Current/previous appointment relationship	Authorized admin only if required
Prescription create/version	No	No	Assigned doctor + record access	No
Doctor approval / specialization	No	No	No	Admin only
Reports	No	No	Own schedule views only	Admin only

5. Proposed Endpoint Catalogue

The proposed catalogue contains 57 operations across authentication, patient, doctor discovery/workspace, appointments, medical records, prescriptions, notifications and administration. The table is a design contract, not evidence that the routes exist in code.

Authentication

ID	Method	Route	Role	Purpose	Contract	Responses
AUTH-001	POST	/api/v1/auth/register/patient	Public	Register a patient account and patient profile.	PatientRegistrationRequest → PatientRegistrationResponse	201, 400, 409
AUTH-002	POST	/api/v1/auth/register/doctor	Public	Submit a doctor registration; account remains Pending until Admin approval.	DoctorRegistrationRequest → DoctorRegistrationResponse	201, 400, 409
AUTH-003	POST	/api/v1/auth/login	Public	Authenticate a user and issue approved API credentials.	LoginRequest → TokenResponse	200, 400, 401, 423
AUTH-004	POST	/api/v1/auth/refresh	Public*	Exchange a valid refresh credential for a new access token.	RefreshTokenRequest → TokenResponse	200, 401
AUTH-005	POST	/api/v1/auth/logout	Authenticated	Revoke the current refresh/session credential where implemented.	LogoutRequest → OperationResult	204, 401
AUTH-006	GET	/api/v1/auth/me	Authenticated	Return the current user identity, role and permitted profile summary.	— → CurrentUserResponse	200, 401
AUTH-007	POST	/api/v1/auth/forgot-password	Public	Start the password reset workflow without revealing account existence.	ForgotPasswordRequest → OperationResult	202, 400
AUTH-008	POST	/api/v1/auth/reset-password	Public	Reset password using a valid one-time reset token.	ResetPasswordRequest → OperationResult	200, 400, 410

Patients

ID	Method	Route	Role	Purpose	Contract	Responses
PAT-001	GET	/api/v1/patients/me	Patient	Retrieve the authenticated patient profile.	— → PatientProfileResponse	200, 401, 403
PAT-002	PUT	/api/v1/patients/me	Patient	Update the authenticated patient profile, medical history and allergies.	PatientProfileUpdateRequest → PatientProfileResponse	200, 400, 401
PAT-003	GET	/api/v1/patients/me/appointments	Patient	Retrieve the patient's upcoming and past appointments.	Query: status, from, to, page, pageSize → PagedAppointmentResponse	200, 401
PAT-004	GET	/api/v1/patients/me/medical-records	Patient	Retrieve only the authenticated patient's medical records.	Query: page, pageSize → PagedMedicalRecordResponse	200, 401
PAT-005	GET	/api/v1/patients/me/prescriptions	Patient	Retrieve the patient's prescription history.	Query: page, pageSize → PagedPrescriptionResponse	200, 401

Doctor Discovery

ID	Method	Route	Role	Purpose	Contract	Responses
DOC-001	GET	/api/v1/doctors	Public	Search approved and active doctors with filters and paging.	Query: name, specializationId, location, availableFrom, availableTo, page, pageSize → PagedDoctorResponse	200, 400
DOC-002	GET	/api/v1/doctors/{doctorId}	Public	Retrieve a public profile of an approved and active doctor.	Path: doctorId → DoctorPublicProfileResponse	200, 404
DOC-003	GET	/api/v1/doctors/{doctorId}/availability	Public	Retrieve available appointment slots within a requested date range.	Query: from, to, appointmentType? → AvailabilityResponse	200, 400, 404

Doctor Workspace

ID	Method	Route	Role	Purpose	Contract	Responses
DOC-004	GET	/api/v1/doctors/me/appointments	Doctor	Retrieve the authenticated doctor's appointments.	Query: from, to, status, page, pageSize → PagedAppointmentResponse	200, 401, 403
DOC-005	GET	/api/v1/doctors/me/schedules	Doctor	Retrieve the doctor's working-hour schedule entries.	— → DoctorScheduleResponse[]	200, 401, 403
DOC-006	PUT	/api/v1/doctors/me/schedules	Doctor	Replace or update working-hour schedule entries after conflict validation.	DoctorScheduleUpdateRequest → DoctorScheduleResponse[]	200, 400, 409
DOC-007	POST	/api/v1/doctors/me/vacations	Doctor	Create a vacation period if it does not overlap Confirmed appointments.	DoctorVacationRequest → DoctorVacationResponse	201, 400, 409
DOC-008	PUT	/api/v1/doctors/me/vacations/{vacationId}	Doctor	Update an existing vacation period after business-rule validation.	DoctorVacationRequest → DoctorVacationResponse	200, 400, 404, 409
DOC-009	DELETE	/api/v1/doctors/me/vacations/{vacationId}	Doctor	Remove an eligible vacation period.	— → —	204, 404, 409

Appointments

ID	Method	Route	Role	Purpose	Contract	Responses
APT-001	POST	/api/v1/appointments	Patient	Create a Pending appointment after validating doctor approval, slot and concurrency.	CreateAppointmentRequest → AppointmentResponse	201, 400, 401, 403, 404, 409
APT-002	GET	/api/v1/appointments/{appointmentId}	Authenticated	Retrieve an appointment only if the caller is authorized for that resource.	Path: appointmentId → AppointmentResponse	200, 401, 403, 404
APT-003	GET	/api/v1/appointments/me	Authenticated	Retrieve appointments belonging to the authenticated patient or doctor.	Query: roleView, status, from, to, page, pageSize → PagedAppointmentResponse	200, 401, 403
APT-004	POST	/api/v1/appointments/{appointmentId}/cancel	Patient/Admin	Cancel an eligible appointment and release its slot.	CancelAppointmentRequest → AppointmentResponse	200, 400, 401, 403, 404, 409
APT-005	POST	/api/v1/appointments/{appointmentId}/reschedule	Patient	Move an eligible appointment to a free slot after applying the 24-hour rule.	RescheduleAppointmentRequest → AppointmentResponse	200, 400, 401, 403, 404, 409
APT-006	POST	/api/v1/appointments/{appointmentId}/confirm	Doctor	Confirm a Pending appointment assigned to the authenticated doctor.	— → AppointmentResponse	200, 401, 403, 404, 409

ID	Method	Route	Role	Purpose	Contract	Responses
APT-007	POST	/api/v1/appointments/{appointmentId}/reject	Doctor	Reject a Pending appointment; a rejection reason is required.	RejectAppointmentRequest → AppointmentResponse	200, 400, 401, 403, 404, 409
APT-008	POST	/api/v1/appointments/{appointmentId}/complete	Doctor	Mark an eligible visit Completed under the approved visit workflow.	CompleteAppointmentRequest? → AppointmentResponse	200, 400, 401, 403, 404, 409
APT-009	POST	/api/v1/appointments/{appointmentId}/no-show	Doctor	Mark an eligible appointment No-show if the final no-show policy is approved.	NoShowRequest? → AppointmentResponse	200, 400, 401, 403, 404, 409

Medical Records

ID	Method	Route	Role	Purpose	Contract	Responses
MED-001	POST	/api/v1/appointments/{appointmentId}/medical-record	Doctor	Create the single medical record allowed for an eligible appointment.	CreateMedicalRecordRequest → MedicalRecordResponse	201, 400, 401, 403, 404, 409
MED-002	GET	/api/v1/medical-records/{medicalRecordId}	Authenticated	Retrieve a medical record after patient/doctor ownership authorization; audit the access.	Path: medicalRecordId → MedicalRecordResponse	200, 401, 403, 404
MED-003	GET	/api/v1/patients/{patientId}/medical-records	Doctor/Admin	Retrieve a patient history only when the doctor has a qualifying appointment relationship or Admin permission.	Path: patientId; Query: page, pageSize → PagedMedicalRecordResponse	200, 401, 403, 404

Prescriptions

ID	Method	Route	Role	Purpose	Contract	Responses
RX-001	POST	/api/v1/medical-records/{medicalRecordId}/prescriptions	Doctor	Issue a new immutable prescription version with medication items.	CreatePrescriptionRequest → PrescriptionResponse	201, 400, 401, 403, 404, 409
RX-002	POST	/api/v1/prescriptions/{prescriptionId}/versions	Doctor	Create a new version instead of editing an issued prescription.	CreatePrescriptionVersionRequest → PrescriptionResponse	201, 400, 401, 403, 404, 409
RX-003	GET	/api/v1/prescriptions/{prescriptionId}	Authenticated	Retrieve a prescription if the user is authorized through its medical record.	Path: prescriptionId → PrescriptionResponse	200, 401, 403, 404
RX-004	GET	/api/v1/medical-records/{medicalRecordId}/prescriptions	Authenticated	Retrieve prescription versions for an authorized medical record.	Path: medicalRecordId → PrescriptionResponse[]	200, 401, 403, 404

Notifications

ID	Method	Route	Role	Purpose	Contract	Responses
NOT-001	GET	/api/v1/notifications/me	Authenticated	Retrieve notifications belonging to the authenticated user.	Query: unreadOnly, page, pageSize → PagedNotificationResponse	200, 401
NOT-002	PATCH	/api/v1/notifications/{notificationId}/read	Authenticated	Mark an owned notification as read.	Path: notificationId → NotificationResponse	200, 401, 403, 404

ID	Method	Route	Role	Purpose	Contract	Responses
NOT-003	PUT	/api/v1/notifications/preferences	Patient	Update channel preferences if the storage model is approved. Optional schema extension.	NotificationPreferencesRequest → NotificationPreferencesResponse	200, 400, 401, 409

Administration

ID	Method	Route	Role	Purpose	Contract	Responses
ADM-001	GET	/api/v1/admin/doctors/pending	Admin	List doctor registrations awaiting approval.	Query: page, pageSize → PagedDoctorResponse	200, 401, 403
ADM-002	POST	/api/v1/admin/doctors/{doctorId}/approve	Admin	Approve a pending doctor registration.	ApproveDoctorRequest? → DoctorAdminResponse	200, 401, 403, 404, 409
ADM-003	POST	/api/v1/admin/doctors/{doctorId}/reject	Admin	Reject a pending doctor registration and optionally store a reason.	RejectDoctorRequest → DoctorAdminResponse	200, 400, 401, 403, 404, 409
ADM-004	PATCH	/api/v1/admin/doctors/{doctorId}/status	Admin	Activate or suspend an approved doctor account.	UpdateDoctorStatusRequest → DoctorAdminResponse	200, 400, 401, 403, 404, 409
ADM-005	GET	/api/v1/admin/patients	Admin	Search and page patient accounts.	Query: search, status, page, pageSize → PagedPatientAdminResponse	200, 401, 403
ADM-006	PATCH	/api/v1/admin/patients/{patientId}/access	Admin	Activate or suspend patient access.	UpdatePatientAccessRequest → PatientAdminResponse	200, 400, 401, 403, 404
ADM-007	GET	/api/v1/admin/specializations	Admin	List specializations for administrative management.	Query: page, pageSize → SpecializationResponse[]	200, 401, 403
ADM-008	POST	/api/v1/admin/specializations	Admin	Create a medical specialization.	CreateSpecializationRequest → SpecializationResponse	201, 400, 401, 403, 409
ADM-009	PUT	/api/v1/admin/specializations/{specializationId}	Admin	Update a medical specialization.	UpdateSpecializationRequest → SpecializationResponse	200, 400, 401, 403, 404, 409
ADM-010	DELETE	/api/v1/admin/specializations/{specializationId}	Admin	Delete a specialization only when referential rules permit it.	— → —	204, 401, 403, 404, 409
ADM-011	GET	/api/v1/admin/appointments	Admin	Search and manage appointments across the clinic.	Query: doctorId, patientId, status, from, to, page, pageSize → PagedAppointmentResponse	200, 401, 403
ADM-012	GET	/api/v1/admin/reports/appointments	Admin	Return appointment trend and status aggregates.	Query: from, to, groupBy → AppointmentReportResponse	200, 400, 401, 403
ADM-013	GET	/api/v1/admin/reports/summary	Admin	Return supported dashboard KPIs and popularity aggregates.	Query: from, to → AdminSummaryResponse	200, 400, 401, 403
ADM-014	GET	/api/v1/admin/reports/revenue	Admin	Return fee-based estimated/recognized revenue summaries only where the data supports them; no payment ledger exists.	Query: from, to, groupBy → RevenueSummaryResponse	200, 400, 401, 403
ADM-015	GET	/api/v1/admin/clinic-settings	Admin	Retrieve clinic-wide settings if ClinicSettings storage is approved. Optional schema extension.	— → ClinicSettingsResponse	200, 401, 403, 404
ADM-016	PUT	/api/v1/admin/clinic-settings	Admin	Update clinic name, default slot length and cancellation policy if implemented. Optional schema extension.	ClinicSettingsUpdateRequest → ClinicSettingsResponse	200, 400, 401, 403

6. Core Request / Response Examples

6.1 Login

POST /api/v1/auth/login — proposed request:

```
{
  "email": "patient@example.com",
```

```
  "password": "<user-supplied-secret>"
}
```

Proposed response (sensitive token values are placeholders):

```
{
  "accessToken": "<short-lived-access-token>",
  "expiresIn": 900,
  "refreshToken": "<opaque-refresh-token>",
  "user": { "userId": 123, "role": "Patient" }
}
```

6.2 Create Appointment

POST /api/v1/appointments — Patient only. The server derives PatientId from the authenticated principal; clients must not be trusted to set another patient id.

```
{
  "doctorId": 42,
  "startDateTime": "2026-11-20T10:30:00+02:00",
  "endDateTime": "2026-11-20T11:00:00+02:00",
  "reason": "Initial consultation",
  "notes": "Optional booking notes"
}
```

Expected behavior: validate doctor approval, working hours, vacation, past-date rules and interval overlap; create a Pending appointment atomically. Return 409 if another request has taken an overlapping slot. The server normalizes timestamps to UTC.

6.3 Reject Appointment

POST /api/v1/appointments/{appointmentId}/reject — assigned Doctor only; appointment must still be Pending.

```
{ "reason": "Doctor unavailable at requested time" }
```

6.4 Create Medical Record

POST /api/v1/appointments/{appointmentId}/medical-record — assigned Doctor only; appointment must be Confirmed or Completed according to the approved workflow; one record per appointment.

```
{
  "symptoms": "Reported symptoms",
  "diagnosis": "Clinical diagnosis",
  "treatmentNotes": "Treatment notes"
}
```

6.5 Issue Prescription

```
{
  "items": [
    { "medicationName": "Medication A", "dosage": "1 tablet", "frequency": "Twice daily", "duration": "5 days",
"instructions": "After meals" }
  ]
}
```

An issued version is immutable. To change medication instructions, create a new prescription version associated with the same MedicalRecordId. Enforce uniqueness of (MedicalRecordId, VersionNumber).

7. Standard Error Contract

```
{
  "type": "https://example.invalid/problems/appointment-conflict",
  "title": "Appointment conflict",
  "status": 409,
  "detail": "The requested time overlaps an existing appointment.",
  "traceId": "<correlation-id>",
  "errors": []
}
```

HTTP	Meaning	Typical MediCare case
400	Bad Request	Malformed DTO or validation error
401	Unauthorized	Missing, invalid or expired API credential
403	Forbidden	Wrong role or resource ownership/doctor relationship failed

HTTP	Meaning	Typical MediCare case
404	Not Found	Identifier does not exist or is not visible under anti-enumeration policy
409	Conflict	Slot overlap, stale Pending state, duplicate medical record, duplicate prescription version
422	Unprocessable Entity (optional convention)	Semantically invalid command if the API adopts this code
429	Too Many Requests	Rate limit triggered, especially auth endpoints
500	Internal Server Error	Unexpected error; return traceId, not stack trace

8. SignalR Contract (proposed)

Item	Proposed definition
Hub	/hubs/notifications
Owner	MediCare.API; Azure SignalR Service used in Default mode
Authentication	Authenticated connection; trusted negotiation; userId comes from validated claims, never client-supplied group ids
Events	AppointmentBooked, AppointmentConfirmed, AppointmentRejected, AppointmentCancelled, AppointmentReminder, DoctorApprovalUpdated
Payload minimization	NotificationId, eventType, appointmentId, status, occurredAtUtc; do not include diagnosis, symptoms or prescription content
Delivery model	Best effort real-time signal plus persisted in-app notification; reconnecting clients should retrieve notifications through REST
Security	Server assigns user-specific routing; reject client attempts to join arbitrary user groups

9. External Provider Integration

Twilio SMS

- API service calls a backend Twilio adapter over HTTPS.
- Use sandbox/test credentials in Staging and production credentials only in Production.
- Queue/retry transient errors with bounded exponential backoff and idempotency/correlation identifiers.
- Never log full message content if it contains patient-sensitive information.

SendGrid Email

- API service calls a backend SendGrid adapter over HTTPS.
- Use verified sender identity and environment-specific templates/configuration.
- Handle provider rejection, throttling and transient failures separately.
- Do not treat an accepted provider request as proof of final delivery unless delivery events are integrated.

10. Database Alignment and Known Gaps

Area	Baseline evidence / gap	Required decision
Identity	The design has Users; standard Identity may create AspNetUsers and related tables.	Choose one authoritative identity key and map Patient/Doctor profile UserId consistently; avoid duplicate logical accounts.
Doctor location	Doctor search requires location, but physical Doctors columns shown do not clearly include location.	Add structured City/ClinicLocation fields or remove location filter from first release.
Schedule slot length	Requirement needs day/from/to/slot length; DoctorSchedules physical schema shows times/IsAvailable but no confirmed slot-length field.	Add SlotLengthMinutes or use an approved ClinicSettings default; document effective-dated behavior.
Appointment reason/notes/type	Requirements include reason/optional notes and appointment types as Should; physical Appointments screenshot does not show all these columns.	Approve nullable schema fields if those features are implemented.

Area	Baseline evidence / gap	Required decision
Overlap concurrency	NFR mentions RowVersion, but RowVersion alone cannot prevent two concurrent inserts with overlapping intervals.	Use RowVersion for updates plus transaction/locking or a slot-reservation constraint that enforces interval overlap prevention.
Medical document storage	No MedicalDocuments table in baseline.	Only add MedicalDocuments metadata plus secure object storage if uploads are included.
Audit trail	NFR requires logging every record open/change; no AuditLogs table in baseline.	Approve tamper-resistant audit storage; do not rely solely on ordinary telemetry.
Notification delivery state/preferences	Notifications baseline has Title, Message, Type, IsRead, CreatedAt; channel/retry/preferences storage is not fully represented.	Extend schema or add outbox/preferences tables only after approval.

11. OpenAPI and Postman

MediCare_OpenAPI.yaml and MediCare_Postman_Collection.json are included as proposed artifacts. They use the same route catalogue as this document. Replace generic DTO placeholders with the actual C# request/response types before implementation is treated as final. Keep the OpenAPI file generated from the API source where possible, to prevent contract drift.

12. API Release Checklist

- Confirm actual routes and DTOs against MediCare.API source code.
- Confirm authentication flow and JWT/cookie design with a security review.
- Confirm role and resource policies using negative tests, not only happy-path tests.
- Confirm request/response schemas and API versioning.
- Confirm every appointment mutation is idempotent or safely conflict-protected.
- Confirm medical record read/write audit events are persisted securely.
- Confirm optional endpoints have approved storage/schema support.
- Generate OpenAPI from the implementation and run contract checks against Postman/Newman.